DogExplorer

Yeganeh Vahabi
LT2319 Dialogue Systems 2

November 2026

1 Introduction

My project is a spoken dialogue system that helps someone who wants a dog, but does not know which breed, find breeds that fit their life. The user says what kind of dog they want (size, how active it should be, how much grooming they can handle, and temperament). The system asks for what is still missing, suggests three matching breeds at a time and says why each one fits. Then it answers follow-up questions about them, such as "does it shed a lot?".

The dialogue manager is rule-based and follows the Information State Update (ISU) approach from SISU. An LLM (Llama 3.1) is used for understanding the user, for answering care questions with retrieval-augmented generation (RAG), and for replying to things the rules cannot handle. The system runs in the browser with Azure speech recognition and synthesis through SpeechState. There is also a text version in the terminal, which I used for development and testing.

2 Pre-existing services and APIs

The main service is TheDogAPI, a free REST API with information about dog breeds. I use two endpoints:

Endpoint	Used for
`GET /v1/breeds`	The whole breed list. It is downloaded once and kept in memory for matching.
`GET /v1/breeds/search?q=name`	Finding a breed the user names, e.g. "Tell me about Beagles".

For each breed the API gives the name, weight and height ranges, life span, a list of temperament words, origin, breed group, a short description and a picture.

The other services the system builds on are:

Service	Role in DogExplorer
SpeechState + Azure	Speech recognition and synthesis in the browser (en-US)
XState	State machine for the spoken turn loop
Ollama `llama3.1`	NLU to JSON, RAG answers, free replies to off-topic turns
Ollama `qwen3-embedding`	384-dimensional embeddings of breed texts and questions
Qdrant	Vector database for retrieval, filtered by breed

The biggest limitation of TheDogAPI is that it has no fields for energy level, grooming, training, apartment living or health, which are exactly the things people want to know when choosing a dog.

3 Data collection

3.1 Scenario

A person who wants a dog, but does not know which breed, describes what they need. The system fills in what is missing and suggests breeds. For example:

U: I'm looking for a small, friendly dog that doesn't need much grooming.

S: How active would you like the dog to be?

Size, temperament and grooming are already given, so activity is the only thing still missing, and that is what the system asks about. After that the user looks at the suggested breeds, asks about them, asks for other options or changes their preferences.

3.2 Collected material

I collected two kinds of material: texts about breeds, which the system uses to match and answer questions, and a set of dialogues that show what the system should handle.

Source	Content	Used for	Amount
TheDogAPI	Structured attributes, description, history	Matching + RAG	630 breeds
Wikipedia	Care sections: temperament, health, grooming, exercise	RAG	59 breeds

The Wikipedia articles are for 59 common breeds, the ones people are most likely to ask about. Each text file has the breed name, the source URL and the license (CC BY-SA 4.0), followed by sections like *Overview*, *Appearance*, *Temperament* and *Health*.

3.3 Distillation and analysis results

When distilling the dialogues, I made the system's turns short enough to listen to. Each system turn does one thing, or gives one piece of feedback and asks one question. From the dialogues I took the patterns the system needs to handle. The dialogues have two phases: first the system *collects preferences*, then the user *explores the results*.

Pattern	Example	Phase
Several preferences in one turn	"A medium, active dog that doesn't need much grooming"	Collecting
Indirect answers	"I go running every morning" (high activity), "nothing too big" (small or medium)	Collecting
No preference	"I don't mind", "skip"	Collecting
Skipping the questions	"Just show me some dogs"	Collecting
Relative answers	"Something smaller"	Both
Grounding	"Okay, small size and very active."	Both
A few options with reasons	"First, Whippet, medium, very active, low grooming"	Exploring
Reference by position, then pronoun	"Tell me about the second one", then "Does it shed?"	Exploring
Asking about a named breed	"Tell me about Beagles"	Both
Care questions	Training, apartment living, health	Exploring
Asking for alternatives	"Anything else?"	Exploring
Changing one's mind	"Actually, larger dogs are okay too"	Exploring
Comparing shown breeds	"Which of these is best with kids?"	Exploring
Small talk	"Hello!"	Both

From these patterns I designed the user intents, the preference slots and the question types:

- **Intents:** `set_preferences`, `show_results`, `more_options`, `select_result`, `ask_about_breed`, `general_question`, `restart`, `help`.
- **Preference slots:** size (small, medium, large), activity (low, medium, high, any), grooming (low, medium, high, any), temperament (friendly, calm, ...), and an operation that says whether new values *replace* or are *added to* the old ones ("larger dogs are okay *too*").
- **Breed question types:** description, size, temperament, lifespan, grooming, exercise, apartment, training, health, other.

The system asks about size first, then activity, then grooming, because size removes the most breeds and is what people mention first.

4 Implementation

The system is written in TypeScript. Every user turn goes through the same steps (Figure 1). The same function, `runTurn`, is used by the speech version, the terminal version and the tests.

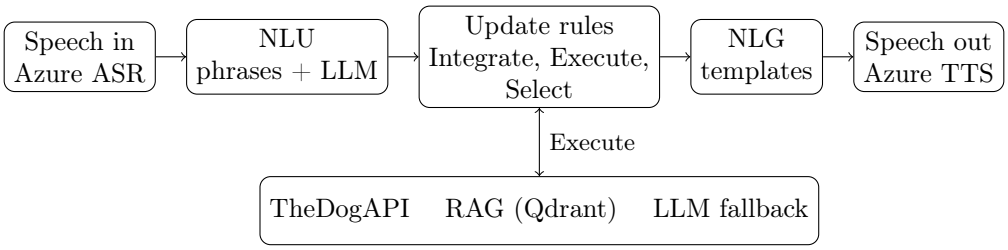


Figure 1: One dialogue turn. Only the Execute rules call outside services.

4.1 Speech

An XState machine drives SpeechState with Azure speech recognition and synthesis. If the user says nothing, the system says "Sorry, I didn't hear you" and repeats what it said last, so the open question is not lost.

4.2 Understanding the user (NLU)

NLU works in two steps:

1. **Fixed phrases.** Common phrases like "I don't mind", "skip", "start over", "help", "anything else" and "show me some dogs" are matched with simple rules. "I don't mind" is filled into the slot that was just asked about. This is faster and more reliable than the LLM for short answers.
2. **LLM parsing.** Everything else goes to Llama 3.1 (temperature 0, JSON mode) with a few-shot prompt. It returns the intent, the slot values, a result number ("the second one"), a breed name and a question type. The prompt includes the last question and the current preferences, so short and relative answers like "something smaller" can be understood. Each field of the output is checked, and values that are not allowed are thrown away.

4.3 Dialogue manager

The NLU result is turned into **user moves**: `answer`, `request_results`, `select`, `ask`, `general`, `restart` and `help`. One utterance can give several moves. For example, "just show me large dogs" gives an `answer` and a `request_results`.

The **information state** contains:

- the user's preferences,
- the breeds shown right now, and the IDs of all breeds shown so far,
- the breed being talked about (what "it" refers to),
- the last question the system asked,
- how many times in a row the system did not understand,
- the phase: `collecting_preferences` or `exploring_results`.

As in SISU, there are three groups of update rules. Each rule has a precondition on the state, and the rules are applied until none fires.

- **Integrate** (9 rules) adds the user's moves to the state. Examples: working out which breed "the second one" is; treating new preferences after the results as a correction, which starts a new search; and confirming only the new sizes when sizes are added ("Okay, medium dogs too").
- **Execute** (3 rules) calls the outside services: breed matching on TheDogAPI data, RAG for care questions, and the LLM fallback.
- **Select** (4 rules) chooses the system moves. `select_ask` only asks the next missing question if more than three breeds still match and the user has not asked for results. `select_next_page` shows breeds that have not been shown yet. `select_reraise_question` asks the open question again after a digression or a misunderstanding.

4.4 Breed matching and estimated traits

Because the API has no activity or grooming fields, the system estimates them:

- **Size** comes from the average weight: under 10 kg is small, 10–25 kg is medium, over 25 kg is large.
- **Activity** comes from words in the temperament and description ("energetic", "athletic" count up; "calm", "lap dog" count down) and from the breed group (Sporting and Herding up, Toy down).
- **Grooming** comes only from the words in front of "coat", "hair" or "fur" in the description ("long", "silky" mean high; "short", "smooth" mean low; "double", "wiry" mean medium). This way "long ears" does not count as a long coat.

Size is a strict filter. Activity and grooming get 2 points for an exact match, 1 for a neighbouring level and 0.5 if the level is unknown. Breeds that are the opposite of what the user wants are removed. Each requested temperament word adds 1 point. When scores are equal, 65 well-known breeds come before rare ones. The estimates are also used to explain each result: "Miniature Schnauzer, small, very active, moderate grooming."

4.5 RAG for breed questions

Indexing. The 689 TheDogAPI and Wikipedia texts are split into chunks of 500 characters with 50 characters overlap, embedded with `qwen3-embedding` and stored in Qdrant together with the breed name and source URL.
Answering. For grooming, exercise, apartment, training, health and other questions, the user's question is embedded and the 4 most similar chunks about the current breed are retrieved. Llama 3.1 answers in one to three sentences using only those chunks. If the chunks do not contain the answer, it must reply `NO_ANSWER` instead of guessing, and the system then gives an answer based on the API data (for example the coat words).

4.6 LLM fallback, repair and NLG

If no rule can handle the user's turn (small talk, general dog questions, comparing the shown breeds), the LLM gets the utterance together with the preferences and the shown breeds and writes a reply. It is told not to make up breed facts. If it does not understand either, the system gives negative feedback ("Sorry, I didn't understand") and asks its open question again. The second time in a row, it also gives example answers.

All other system answers come from templates, one short sentence per system move. Results are numbered ("First, ... Second, ...") so the user can refer to them.

4.7 Problems encountered

- **TheDogAPI lacks the traits users care about.** Activity and grooming are estimated from words, and the estimates are sometimes wrong.
- **Too many matches.** The API has more than 600 breeds, many of them rare, so after one or two questions there are still hundreds of matches. Ranking well-known breeds first helps only partly.
- **LLM NLU is slow and unstable on short answers.** This is why fixed phrases are matched first and the last question is given to the LLM as context. One turn can still need up to three LLM calls (NLU, RAG, fallback).
- **Speech.** Speech recognition often gets breed names like "Vizsla" or "Shih Tzu" wrong, which breaks the name search, and the long API descriptions are tiring to listen to.
- **Ollama from the browser.** The web version calls Ollama directly. This works for a local demo, but a real deployment would need a backend.

5 Future work

- **Test with real users.** Record real dialogues and see where the NLU and the questions fail, and measure whether people find a breed they like, in how many turns and how often repair is needed.
- **Better trait data.** Use rated energy and grooming levels instead of word estimates, for example by letting the LLM rate each breed once from the RAG texts.
- **Comparing breeds.** Make comparisons like "Which is calmer, the Beagle or the Whippet?" a proper dialogue move, answered from the API temperament data.
- **More slots and more texts.** Ask about children, other pets and experience with dogs, add Wikipedia texts for more breeds, and ask the question that removes the most breeds first.

- **Better ranking after corrections.** Keep the history of shown breeds when a preference is widened, so the newly allowed breeds come first.
- **Spoken output and speech recognition.** Shorten the long descriptions, show the breed pictures on screen, and use fuzzy matching or a phrase list for breed names.

A Code

File	Purpose
`src/speechMain.ts`, `src/dm.ts`	Browser UI and the XState/SpeechState speech loop
`src/main.ts`	Text version in the terminal
`src/processDialogue.ts`	`runTurn`: runs the update rules for one turn
`src/nlu.ts`	Fixed phrases and LLM-based NLU with output checking
`src/moves.ts`	User and system moves; NLU result to user moves
`src/dialogueState.ts`, `src/dialogueManager.ts`	Information state and which question to ask next
`src/rules.ts`	Integrate, Execute and Select rules
`src/breedMatcher.ts`, `src/commonBreeds.ts`	Trait estimates and breed ranking
`src/breedAnswers.ts`, `src/nlg.ts`	Answers from the API data and templates
`src/rag.ts`, `rag/main.ts`	RAG answering and the indexing tool
`src/fallback.ts`	LLM fallback replies
`src/dogApi.ts`	TheDogAPI client

for running: start Ollama with `llama3.1` and `qwen3-embedding`, start Qdrant (`rag/docker-compose.yml`), put the TheDogAPI and Azure keys in `src/credentials.ts`, and run `npm run dev` (speech) or `npm run terminal` (text).